GAME / GRAPHICS PROGRAMMER

描画の仕組みから、
ゲームの手応えをつくる。

専門学校2年のニックです。C++ / DirectX 11 / HLSL で、描画パイプラインからエフェクト、ゲームロジックまでを個人で制作しています。



BreaveChicken のタイトル画面（実機キャプチャ）

掲載作品

3 作品

ゲーム2作品とツール1作品

個人制作の期間

3 か月 ×3

BreaveChicken / TrashBomb / シェーダエディタ

描画パイプライン

6 段階

ScenePass から GPU Particle まで

フレームレート改善

180 fps

BreaveChicken のゲーム開始時。修正前は100fps（1280×720）

STAGE 01 / WORKS

制作作品

代表作は BreaveChicken。描画パイプラインとGPUパーティクルを自作し、アクションの手応えにつなげました。



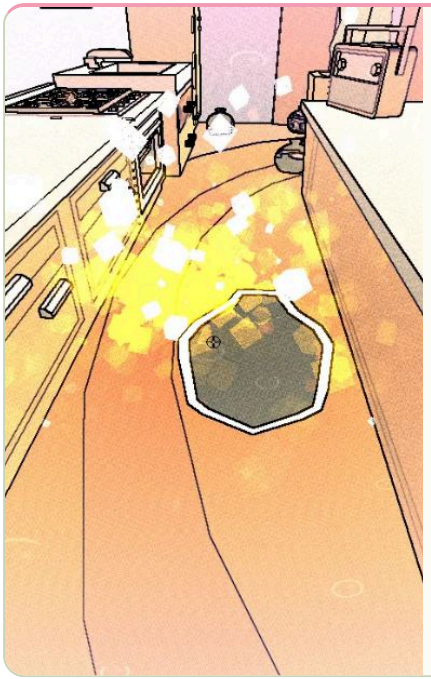
01 ゲーム／アクション 代表作

BreaveChicken ブレイブチキン

ニワトリが5匹のひよこを連れて家に帰るアクション。キツネからひよこを守りながら戦う焦燥感と、スピーディーな爽快感が見どころです。

期間	3か月	人数	個人制作
担当	全工程	環境	C++ / DirectX 11
時間	約10分		

Deferred Shading / G-buffer / Compute Shader / GPU Particle / HLSL



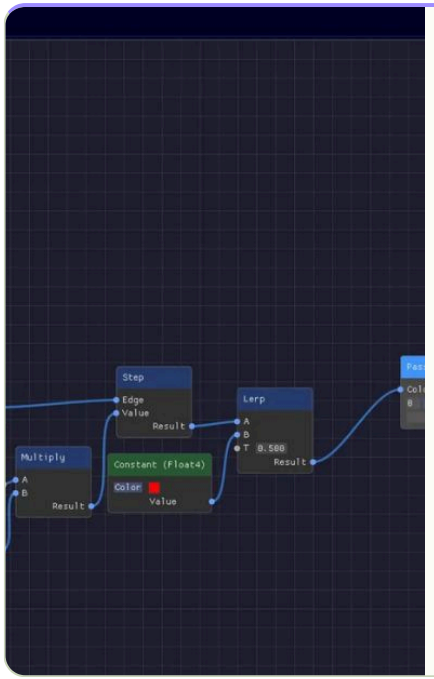
02 ゲーム／ミニゲーム

TrashBomb

日々の鬱憤が夢の中でゴミになる。ほうきで叩いてゴミを連鎖爆発させ、3分間のスコアを競います。

期間	**3か月**	人数	**個人制作**
担当	**全工程**	環境	**C++ / DirectX 11**
時間	**約3分**		

`Edge Detection` `Sobel` `Toon Shading` `Transition`



03 エディタ（ツール）／シェーダ作成

シェーダエディタ

ノードを組むだけでHLSLが自動生成され、書き出したJSONをゲーム側が読み込んで反映するエディタ。BreaveChicken と連携しています。

期間	**3か月**	人数	**個人制作**
担当	**全工程**	環境	**C++ / DirectX 11**

`Node Graph` `HLSL` `JSON Asset` `Multi Pass` `Hot Reload`

STAGE 02 / TECH

技術ハイライト

課題、実装、成果の順にまとめています。性能の数値は実測できたものだけを載せる方針です。

EFFECTS / COMPUTE

GPUパーティクルシステム

粒子の挙動計算を GPU に集約

BreaveChicken で使用。粒子数などの計測値は準備中

Emit 要求 → Compute Shader → パラメータで一括制御 → ACES

課題 表現力を上げると、CPU側の処理負荷が制約になる。

実装 1回のEmitを1つの再生単位として扱い、粒子の動きをCompute Shaderで一括制御。

RENDERING ARCHITECTURE

描画パイプライン設計

G-buffer を軸に 6段階で構成

BreaveChicken で使用。構成図つき

ScenePass → MaterialPass → 各種レンダリング → AA・調整 → PostEffect → 特殊エフェクト

課題 見た目を統一して調整しやすくしたい。ゲームシステムを描画の内部に依存させたくない。

実装 遅延シェーディング、パス分離、カリング・LOD・インスタンシング、エディタ連携。

PROFILING / OPTIMIZATION

GPU負荷の最適化

フレームレート（ゲーム開始時）

100fps → 180fps

1280×720。修正前のコミットと現在のビルドを、同じPC・同じ場面で計測

1フレームの時間

10.0ms → 約5.6ms

fps から換算（約44%短縮）

BreaveChicken で実施。ゲーム開始時・1280×720・同じPC で修正前後を計測

キャプチャ → パスごとに計測 → 原因を特定 → 削減 → 再計測

課題 ゲーム開始時のフレームレートが 100fps 前後で、描画負荷が高かった。

実装 RenderDoc で重いパスを特定し、ポリゴン数・ノード数・シャドウのステップ数を削減。

成果 CPU負荷を抑えつつ大量の粒子を使い、攻撃やヒ

GPU-DRIVEN RENDERING

GPUカリングの実装と技術判断

実装・検証して 現在も使用中

BreaveChicken で使用中。対象規模はオブジェクト数千個未満。性能比較値は未計測

StructuredBuffer → 視錐台6平面 → コンパクション → Indirect Draw

課題 同種のオブジェクトをまとめ、GPUの並列性で描画を効率化できないか。

実装 GPU側で可視判定・データ整理・間接描画引数の生成まで行い、影の視点でも選別。

判断 この規模では必須ではないと分かったが、問題なく動いており作り直す理由もないため、そのまま使っている。

成果 パラメータを実行中に調整でき、ゲーム側は演出

NPR / POST PROCESS

エッジシェーダ

色・法線・ID の 3種類で輪郭を作る

TrashBomb で使用

Color Edge → Normal Edge → ID Edge → Sobel 3×3 → Max で合成

課題 3Dモデルをそのまま描くと、目指していた2Dアニメ調に対して画面が平坦に見えた。

実装 G-buffer の色・法線・ID から 3×3 の Sobel でエッジを検出し、個別の強度を掛けて Max で合成。

成果 平坦だった画面に立体感とメリハリが出て、目指していたアニメ調のビジュアルに近づいた。

成果 ゲーム開始時のフレームレートが 100fps から

TOOL / VFX

Effect Editor



シェーダエディタの機能。BreaveChicken のエフェクト制作に使用

パラメータ編集 → プレビュー → JSON で保存 → ゲームで再生

課題 エフェクトを作るたびにプログラムを変えていては、制作と調整に時間がかかる。

実装 パーティクルとトレイルを、パラメータ操作で新規作成・再編集できる2つのエディタを作り、設定を JSON アセットで管理。

成果 BreaveChicken のパーティクル23種・トレイル10種を、このエディタで作って管理している。

LOOK DEVELOPMENT

夜の森のルック調整

6段階の調整で
雰囲気と視認性を両立

BreaveChicken のステージで実施。各段階の画面写真つき

暗く → コントラスト → 色調 → フォグ・配置 → 密度・道 → 発光

課題 夜の森らしく暗くすると、プレイヤーや敵が背景に埋もれて見えにくくなる。

実装 明るさ・色調・コントラスト・フォグ・配置・発光を1つずつ見直し、画面で確かめて次の課題を探した。

成果 神秘的で少し不気味な夜の森を保ちつつ、ニワトリ・ひよこ・敵を見分けられる画面になった。

描画パイプラインで変わる見た目

BreaveChicken の同じ場面を、パイプライン適用前と適用後で比べられます。



左が描画パイプライン適用前、右が適用後です。境目をドラッグするか、左右キーで動かせます。ほぼ同じ地点から撮影していますが、経過時間や敵の位置は異なるため、厳密な比較ではありません。

STAGE 03 / ABOUT

自己紹介

ニック　専門学校2年

探求心が強く、授業の範囲にとどまらず独学でも理解を深めています。ゲームプログラミング、シェーダプログラミング、パイプライン構築を得意としています。

BreaveChicken の反省から、見た目を整えるだけでなく、G-buffer の持ち方やメモリ帯域まで含めて描画コストを設計することを次の課題にしています。

志望職種	ゲームプログラマ／グラフィックスプログラマ
言語・ツール	C++ / DirectX 11 / HLSL / Git
得意分野	ゲームプログラミング／シェーダプログラミング／パイプライン構築

STAGE 04 / SKILLS

スキル

各スキルに、使った作品や技術解説へのリンクを付けています。

実装し説明できる 面接で設計を語れる　実装した 作品で使用　学習中 学習段階

言語・API

C++

実装し説明できる　根拠：BreaveChicken

DirectX 11

実装し説明できる　根拠：描画パイプライン設計

HLSL

実装し説明できる　根拠：エッジシェーダ

グラフィックス

遅延シェーディング／G-buffer

実装し説明できる　根拠：描画パイプライン設計

Compute Shader

実装し説明できる　根拠：GPUパーティクルシステム

カリング・LOD・インスタンシング

実装した　根拠：GPUカリングの実装と技術判断

NPR（エッジ検出・トゥーン）

実装した　根拠：エッジシェーダ

ルック調整（色調・コントラスト・フォグ・発光）

実装した　根拠：夜の森のルック調整

ツール・開発

ノードベースのツール開発（HLSL 生成・JSON 出力）

実装した　根拠：シェーダエディタ

エフェクト制作ツール（パーティクル・トレイル）

実装した　根拠：Effect Editor

エディタ連携（パラメータ調整・自動保存）

実装した　根拠：描画パイプライン設計

GPU プロファイリング（RenderDoc）

実装した　根拠：GPU負荷の最適化

Visual Studio

実装した　根拠：BreaveChicken

Git

実装した　根拠：GitHub 準備中

STAGE 05 / CONTACT

連絡先

作品や技術についてのご質問、選考のご連絡はメールでお願いします。

メール bouniku@meru.dayo.jp アドレスをコピー

作品・技術の詳細



開始時の目的表示。HUD には回収したひよこの数、経過時間、ゲージ、操作ガイドを配置しています。

BreaveChicken ブレイブチキン

ニワトリが5匹のひよこを連れて、無事に家へ帰るゲームです。帰り道では、ひよこを狙うキツネが行く手を阻みます。

タイトルは Breath（息）と Brave（勇敢）を合わせた造語です。ひよこを守って戦う勇敢な鶏という意味に、ゲームの中心である肺活量ゲージを重ねています。

担当
全工程（個人制作・3か月）

環境
C＋＋ / DirectX 11 / Visual Studio / Windows 11

ルール
ひよこ全員が家に着けばクリア。ニワトリの体力が尽きるとゲームオーバー。

プレイ時間
1プレイ約10分

工夫
守りの回避が攻めの必殺技につながる、肺活量ゲージの循環。

学び
見た目だけでなく、GPUのメモリ帯域まで含めたコスト設計が必要だと気づいた。

肺活量ゲージと必殺技「鳴き声」

01　敵の攻撃をちょうどのタイミングで回避すると、肺活量ゲージが溜まる。

02　ゲージが Max になると、必殺技「鳴き声」が使える。

03　鳴き声を受けた敵はひるんで動きを止め、氷のように固まる。

04　固まった敵に与えるダメージは倍になる。

凍結の演出は GPUパーティクルとポストエフェクトの組み合わせです。ゲームシステムは発動に合わせてエフェクトを呼ぶだけで、描画の内部処理には関与しません。

描画負荷の調整

描画が重く感じたため、RenderDoc でフレームをキャプチャし、パスごとの負荷を見て原因を絞り込みました。

- 高ポリゴンのモデルを大量に描画していた → モデルのポリゴン数を削減
- ステージのオブジェクトのノード数が多かった → ノード数を削減
- ボリューメトリックと PBR のシャドウが特に重かった → ステップ数を削減

ゲーム開始時のフレームレートは 100fps から 180fps になりました（1280×720、修正前後を同じ PC で計測）。

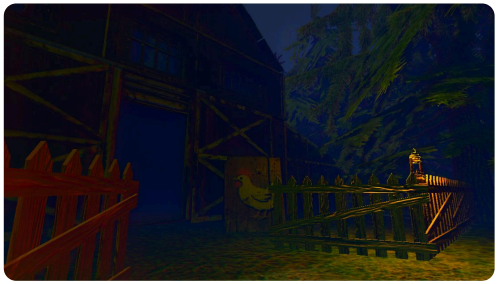
技術アピール

描画パイプライン設計　GPUパーティクルシステム　GPU負荷の最適化　夜の森のルック調整　シェーダエディタ

ゲーム画面



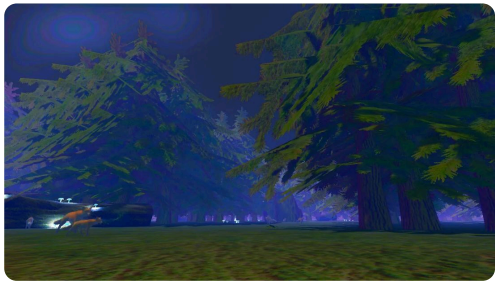
守る対象のひよこ。足元の光と、背景で光るキノコで夜の森に目印を作っています。



ゴールの農場。ランタンの暖色光を、夜空の寒色と対比させています。



ひよこを狙う敵のキツネ。目を赤く光らせ、暗い森でも見つけやすくしています。



帰り道の森。遠くほど青紫に沈ませて、奥行きと夜の空気を出しています。

操作

操作	キーボード・マウス	コントローラー
移動	W / A / S / D	左スティック
視点	マウス	右スティック
攻撃	左クリック	X ボタン
強攻撃	右クリック	Y ボタン
回避	左 Shift	A ボタン
必殺技（鳴き声）	Space	B ボタン

反省と次回への改善

GPU側の処理負荷を十分に考えずに作っており、不要な G-buffer まで生成していました。次は用途に合わせて小さいレンダーターゲットを使い、UAV を活用してメモリ帯域の消費を抑えます。

制作の動機

誰が見ても目に留まる作品を作りたいと考え、凶暴さと親しみやすさを併せ持つニワトリを主人公にしました。

▼ **使用素材・ライセンス** 19件

素材	作者	ライセンス	備考
モデル Chicken (animated)	Čestmír Dammer	CC0 1.0	改変あり。テクスチャは Rick Hoppmann の写真をもとに作成されたもの
モデル Animated Chick	FourthGreen	CC BY 4.0	改変あり
モデル Fox	Alexei Ostapenko	CC BY 4.0	改変あり
モデル Farm (low poly)	Avmifi	CC BY 4.0	改変あり
モデル Fence pack - (low poly, game ready)	TampaJoey	CC BY 4.0	改変あり
モデル Copper Kerosene Lamp	SilverSister	CC BY 4.0	改変なし
モデル Pine Trees Pack | PS1 Low Poly	Vova Truestory	CC BY 4.0	改変あり
モデル low poly forest pack, low poly environment pack	Ragat Vdoo Kaf	CC BY 4.0	改変あり
モデル Low Poly Shrub - Small Texture	Natural_Disbuster	CC BY 4.0	改変なし
モデル Foliage	Daemon	CC BY 4.0	改変なし
モデル Aerial Grassy and Rocky Mountain 1	Mike.xx	CC BY 4.0	改変あり
モデル Free Rock Assets Pack	JonhGillessen	CC BY 4.0	改変あり
モデル Gravel Ground Module Scan Dry 2	Per's Scan Collection	CC BY 4.0	改変あり
モデル Large tree log	Julian Malik	CC BY 4.0	改変あり
モデル Log	Arkikon	CC BY 4.0	改変あり
モデル 看板（AI生成）	Tripo AI	CC BY 4.0	改変なし。Tripo AI の無料プランで生成。ツールの規約により非商用での利用に限る
モデル キノコなど（AI生成）	Tripo AI	CC BY 4.0	改変あり。Tripo AI の無料プランで生成。ツールの規約により非商用での利用に限る
効果音 効果音ラボの効果音（複数）	効果音ラボ	効果音ラボ 利用規約	クレジット表記は任意。素材ファイルそのものの再配布は禁止

素材	作者	ライセンス	備考
BGM **BGMer の楽曲**	BGMer	BGMer 利用規約	クレジット表記は任意。販売・配信リリースは禁止



ゴミが連鎖して爆発した瞬間。右上の「5chain!」が連鎖数、左下がスコアです。

TrashBomb

夢の世界に散らばったゴミを、ほうきで叩いて吹き飛ばし、まとめて処理するゲームです。ゴミが連鎖して爆発する爽快感が売りです。

担当
全工程（個人制作・3か月）

環境
C＋＋ / DirectX 11 / Visual Studio / Windows 11

ルール
制限時間3分。連鎖させるほどスコアが高くなる。

操作
WASD で移動、マウスで視点、左クリックでほうきを振る。

工夫
色・法線・ID の3種類のエッジを組み合わせた輪郭線。

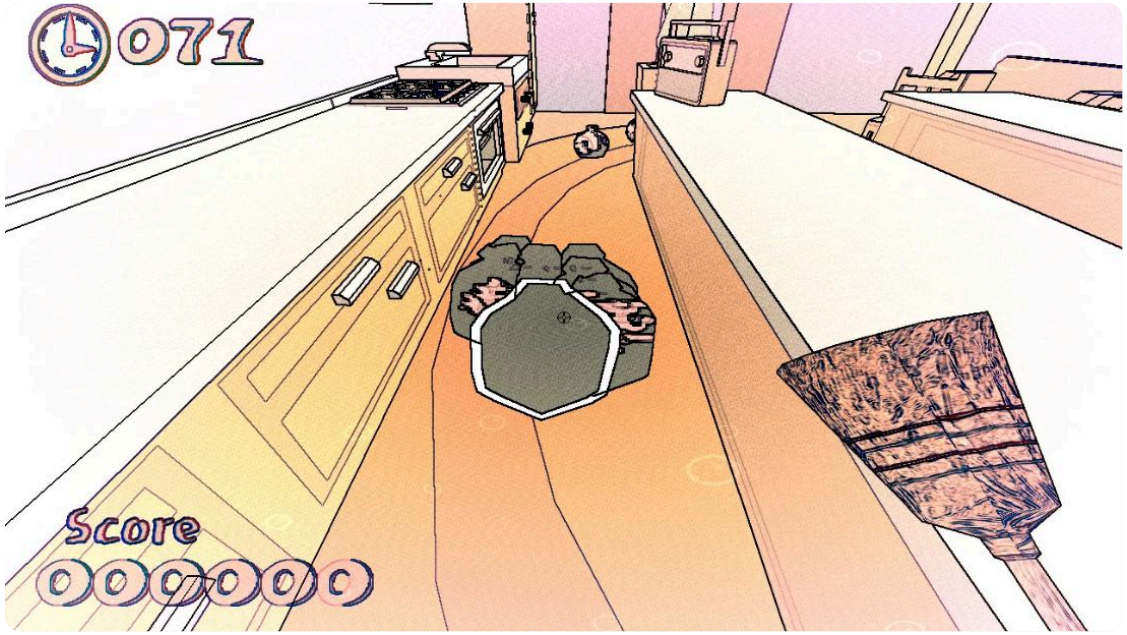
① ゴミの連鎖爆発

01 ゴミが爆発したとき、範囲内のほかのゴミを「爆発予約状態」にする。

02 予約状態のゴミは、数フレーム後に順番に爆発する。

03 爆発したゴミがさらに範囲内のゴミを予約し、連鎖が続く。

連鎖が続くほどスコアが上がるため、この仕組みが「3分間でどれだけ爆発させられるか」というゲームの目的に直結しています。

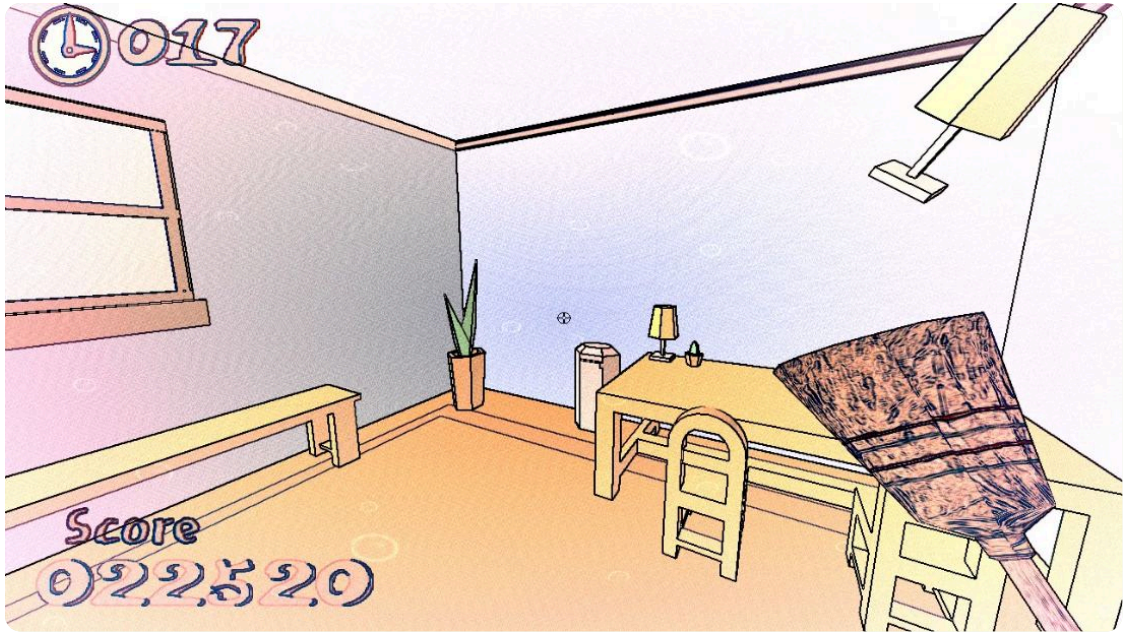


爆発させる前のゴミの山。ほうきで叩いて吹き飛ばし、まとめて連鎖させます。

② エッジシェーダ

`Edge Detection` `Sobel` `G-buffer`

2Dアニメ調の見た目を目指し、G-buffer の色・法線・オブジェクト識別の3種類の情報から輪郭線を作っています。3×3 の Sobel フィルタで検出し、それぞれの強さを調整して Max で合成します。仕組みと改善の流れは技術ハイライトで詳しく説明しています。



エッジシェーダとトゥーンシェーダを適用した部屋。家具どうしの重なりや壁の境目にも輪郭線が入っています。

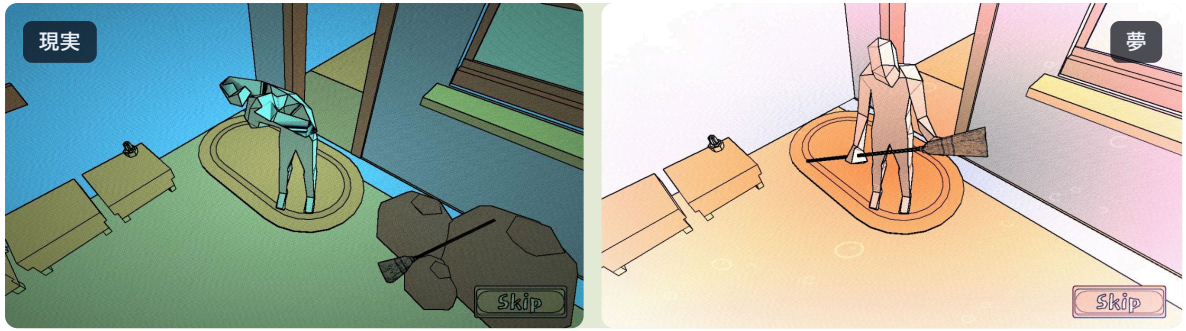
エッジシェーダ

③ トゥーンシェーダ

影を出さずに平坦な見た目にし、夢の世界の非現実感を出すために NPR を採用しました。

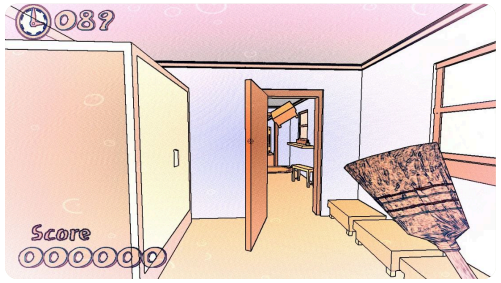
④ ワイプトランジション

シーンを固定してテクスチャにし、それをトランジション用の画像として使うことで、夢と現実の曖昧な境目を表す画面遷移を作っています。

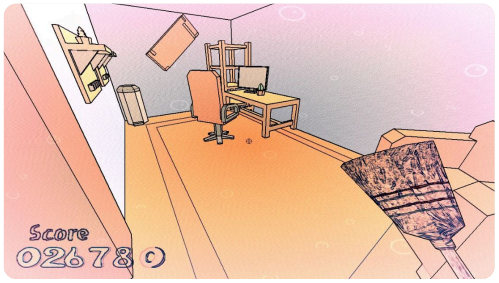


左が現実の部屋、右が夢の中の同じ部屋です。境目をドラッグするか、左右キーで見比べられます。人物のポーズは場面によって異なります。

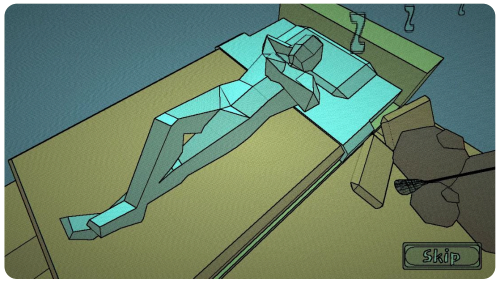
ゲーム画面



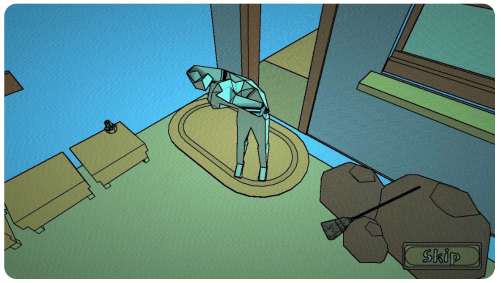
プレイ画面。左上に残り時間、左下にスコア、右下に手に持ったほうきを表示しています。



夢の中の部屋。家具が宙に浮いていて、現実から離れた空間を表しています。



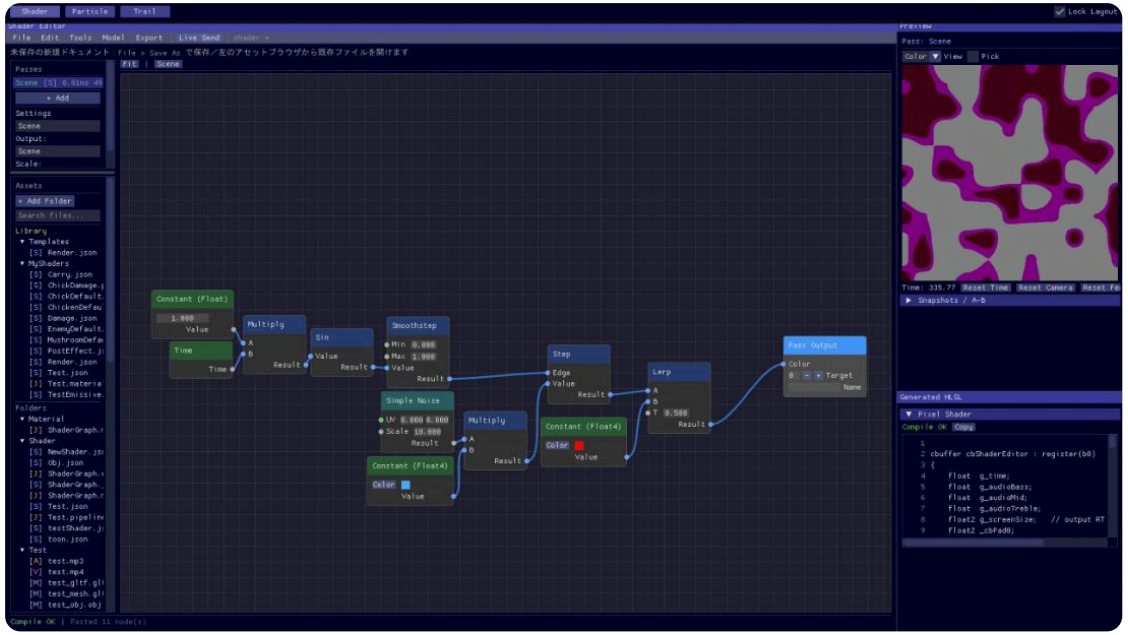
現実の世界の演出シーン。ベッドで眠る場面です。



現実の世界の演出シーン。夢の世界とは色調を大きく変えています。

▼ **使用素材・ライセンス** 5件

素材	作者	ライセンス	備考
モデル **Furniture Pack**	Kenney	CC0 1.0	改変あり
モデル **Very Low Poly Human**	Clint Bellanger	CC0 1.0	改変あり
モデル **inventory items（ほうき）**	sandsound	CC0 1.0	改変なし
効果音 **効果音ラボの効果音（複数）**	効果音ラボ	効果音ラボ 利用規約	クレジット表記は任意。素材ファイルそのものの再配布は禁止
BGM **BGMer の楽曲**	BGMer	BGMer 利用規約	クレジット表記は任意。販売・配信リリースは禁止



左にパスとアセットブラウザ、中央にノードグラフ、右にプレビューと生成された HLSL を並べています（クリックで拡大）。

シェーダエディタ

シェーダノードを組むことで、HLSL を自動で作成するエディタです。ゲームに組み込むときは、JSON 形式で出力したアセットをゲーム側が読み込むだけで扱えます。

担当
全工程（個人制作・3か月）

環境
C++ / DirectX 11 / Visual Studio / Windows 11

想定ユーザー
シェーダを触る人

連携
BreaveChicken（マテリアル・PS 管理の元になる部分）

出力対象
マテリアル / レンダリング / パーティクル / トレイル

ゲームへの反映
JSON アセットを読み込むだけ。ゲーム側はホットリロードに対応

制作の動機と解決したい課題

シェーダを作るときに、テンプレートを作って使い回せたら楽になると考えました。さらに、HLSL を直接書くよりも、ノードを組むほうが直感的にシェーダを作りやすいと考え、制作しました。

> 解決したい課題：HLSL を直接書くよりも直感的な操作で、楽にシェーダを作成できるようにすること。

主な機能

- ノードでシェーダを組む
- 組んだシェーダをアセット化する
- パーティクルエディタ、トレイルエディタを併用できる
- マルチパスに対応
- プレビュー（途中のノードのプレビューも可能）
- 組んだノードの HLSL コードを確認できる
- アセットブラウザ
- ゲーム側でホットリロードに対応（マテリアルなど）

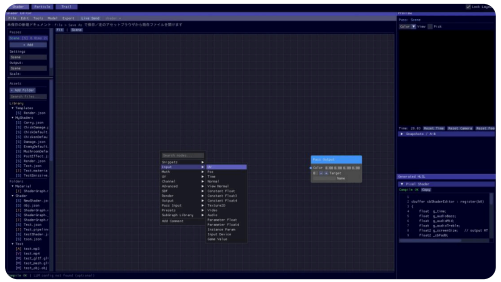
基本的な使い方

01 好きなようにノードを組む（現在は VertexOutput と GbufferOutput が必要）。

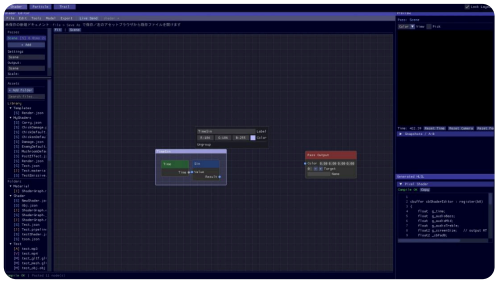
02 組み終わったら Export から、マテリアルなら Asset、レンダリングなら Pipeline を選ぶ。

03 ゲーム側に反映される。

エディタの画面



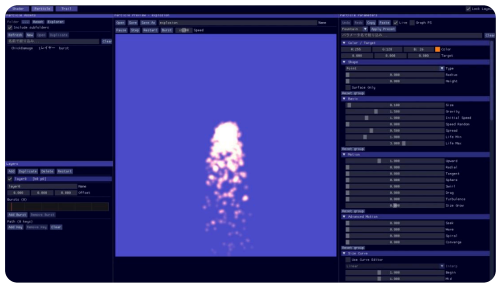
ノード追加メニュー。カテゴリごとに分けて、検索からも追加できます。



ノードのグループ化。よく使う組み合わせ（ここでは Time と Sin）を名前と色つきでまとめられます。

Effect Editor（パーティクル／トレイル）

シェーダエディタには、パーティクルとトレイルをパラメータ操作で作れる Effect Editor も入っています。BreaveChicken のパーティクル23種・トレイル10種は、このエディタで作っています。



Particle Editor。レイヤーとバーストを管理し、プレビューしながらパラメータを調整します。



Trail Editor。斬撃の軌道でプレビューしながら、色や形状、ノイズを調整します。

Effect Editor

① ノードグラフ設計

- 各ノードを識別できるようにしている。
- 各ノードの処理を記述している。
- ノードは入出力や座標などの情報を持ち、特殊なノードに必要な値もまとめて持っている。

② アセットの出力形式

出力時には、次の情報をまとめて JSON として書き出します。

- 名前
- HLSL
- 種類（マテリアルか、パイプラインかなど）
- 頂点レイアウトなど

ファイル名は、.json の前に material などの識別名を付けて、種類を見分けられるようにしています。

BreaveChicken との連携

エディタで作ったマテリアルやレンダリング（パイプライン）を書き出すと、BreaveChicken 側に反映されます。BreaveChicken の描画パイプライン設計にある「マテリアル・PS 管理」の元になる部分です。

> ゲーム実行中にパラメータを調整する「レンダリング設定」の画面は BreaveChicken 側の機能で、このエディタの機能とは別です。

描画パイプライン設計　BreaveChicken

反省と次回への改善

汎用的なものにはなりませんでした。次は次の3点に取り組みます。

- 最終出力が PassOutput だけのものを、SRV としてゲーム側で保存して扱えるようにする。
- エディタで使った画像を圧縮してから、ゲーム側でパスを気にせず使えるようにする。
- 頂点シェーダやコンピュートシェーダにも対応させる。

GPUパーティクルシステム

Compute Shader で粒子の挙動計算を GPU 側に集約し、大量のパーティクルによるエフェクト表現とアクションの手応えを支えています。

Compute Shader | GPU Particle | ACES Tone Mapping

課題

エフェクトの表現力を高めたい一方で、パーティクルを扱う CPU 側の処理が制約になっていました。大量の粒子を使った表現を、ゲーム中でも扱いやすくする必要がありました。

実装

01 1回の Emit 要求を、1つのエフェクトの再生単位として扱う。

02 パーティクルの挙動計算を Compute Shader で実行する。

03 パラメータをもとに、各粒子の動きを GPU 側で一括制御する。

04 生成した表現に ACES トーンマッピングを適用する。

成果

CPU 側の処理負荷を抑えながら、大量のパーティクルをゲーム中に使えるようにしました。攻撃やヒット時の視覚的なフィードバックが強まり、アクションの手応えにつながっています。BreaveChicken の必殺技「鳴き声」の凍結演出にも使っています。

粒子数・同時稼働数・CPU/GPU 処理時間は計測後に掲載します。

描画パイプライン設計

描画処理をパイプラインとして構造化し、世界観の統一、エディタでのリアルタイム調整、ゲームシステムからの演出制御を両立するレンダリング基盤です。

`Deferred Shading` `G-buffer` `HLSL` `Instanced Rendering` `LOD` `Post Effect`



G-buffer を ScenePass で作り、MaterialPass で必要な項目だけ上書きしてから、後続のパスへ順に渡す流れを示した構成図です（クリックで拡大）。

適用前と適用後



左が描画パイプライン適用前、右が適用後です。境目をドラッグするか、左右キーで動かせます。ほぼ同じ地点から撮影していますが、経過時間や敵の位置は異なるため、厳密な比較ではありません。

課題

- ゲーム全体の色彩や質感に統一感を持たせ、見た目を調整しやすくしたい。
- ゲームシステムが描画の内部構造を直接管理せず、必要な演出を使えるようにしたい。

パスの役割

SenePass
全オブジェクトを共通のデフォルトマテリアルで描き、G-buffer を生成する。

MaterialPass
マテリアルを持つオブジェクトだけ G-buffer を上書きする。変更しない項目は ScenePass の値を使い、マテリアルなしはスキップする。

描画の最適化
同じモデルかつ同じマテリアルをグループ化し、視錐台カリングと LOD を適用してインスタンシング描画する。

PostEffect
ゲームシステムから起動する画面演出（ビネット、シーンカラー、ラジアルブラー、色収差など。カメラ演出は除く）。

特殊エフェクト
PostEffect の後に XRay → GhostTrail → Trail → GPU Particle の順で実行する。

使い方
演出を選んでパラメータを設定し、有効にする。不要になれば権限を手放すかフラグを無効にする。

ツール連携（2つの機能を分けて設計）

	レンダリング設定	マテリアル・PS 管理
できること	1パスあたり最大16個のパラメータをエディタで調整し、ゲーム画面にリアルタイム反映。ゲーム終了時に自動保存。	エディタで PS やマテリアルを作ってライブラリに登録し、オブジェクトが名前で参照する。
目的	ソースを書き換えずに見た目を試行錯誤する。	マテリアルを再利用しやすくする。

マテリアルや PS を作るエディタは「シェーダエディタ」として別に掲載しています。

シェーダエディタ

成果

- 描画パラメータの変更が、実行中のゲームにすぐ反映される。
- 調整値が終了時に自動保存されるので、調整作業を続けやすい。
- ゲームシステムは PostEffect を選んで設定するだけで演出を使える。

反省

不要な G-buffer まで生成していたため、次は用途に合わせた小さいレンダーターゲットと UAV の活用でメモリ帯域を抑えます。

GPU負荷の最適化

BreaveChicken の描画負荷を RenderDoc で調べ、重いパスと原因を特定して削減しました。ゲーム開始時のフレームレートは 100fps から 180fps になりました。

RenderDoc GPU Profiling Volumetric PBR Shadow

課題

解像度 1280×720 で、ゲーム開始時のフレームレートが 100fps 前後でした。

調査

RenderDoc でフレームをキャプチャし、パスごと・描画ごとの時間を確認して、原因を次の3つに絞りました。

- 高ポリゴンのモデルを大量に描画していた。
- ステージが高ポリゴンで、オブジェクトのノード数も多かった。
- ボリューメトリックパスと PBR パスの負荷が、ほかのパスより際立って高かった。

対策

01 モデルのポリゴン数を削減した。

02 ボリューメトリックと PBR のシャドウで、ステップ数を減らした。

03 ステージのオブジェクトのノード数を削減した。

結果

	フレームレート	1フレーム（換算）
修正前	100fps	10.0ms
修正後	180fps	約5.6ms

修正前のコミットと現在のビルドを、同じ PC・解像度 1280×720・ゲーム開始時の同じ場面で計測しました。

参考：RenderDoc で見たパスごとの負荷の割合（対策の途中）

パス	割合
全画面パス 13本（ライティング・ボリューメトリック・ポストエフェクトなど）	51%
G-buffer を書き込むパス（5 RT ＋ Depth）	31%
深度のみのパス（Depth-only）	8%
その他（Compute・パーティクル・UI など）	10%

RenderDoc の Duration は処理を1つずつ区切って測る計測用の値で、実際のフレーム時間とは一致しません（GPU のクロック状態でも大きく変わります）。そのため、ms ではなく割合で載せています。2回のキャプチャで割合はほぼ同じでした。

わかったこと

RenderDoc の時間はどのパスが重いかを比べるのに使い、実際の速さは同じ条件でのフレームレートで確かめる、という使い分けが必要だとわかりました。

GPUカリングの実装と技術判断

全インスタンスのデータを GPU に渡し、GPU 側で可視判定・データ整理・間接描画引数の生成まで行う仕組みを実装し、BreaveChicken で使っています。実装後に規模と照らして必要性を検証し、使い続けるかどうかを判断しました。

`StructuredBuffer` `Compute Shader` `GPU Culling` `Instance Compaction` `Indirect Drawing` `LOD`

着想

オブジェクトを1つずつ処理するより、同じ種類をまとめて扱う方が効率的ではないかと考えました。大量のデータを並列に処理できる GPU の特性を活かし、可視判定と描画データの整理を GPU 側で試しました。

実装の流れ

01 全インスタンスのデータを StructuredBuffer で GPU へ転送する。

02 オブジェクト種別と DrawType を基準にグループ化する。

03 Vertex Clustering（空間グリッドの同じセルの頂点を平均位置に集約）で LOD を作る。

04 視錐台の6平面で判定し、完全に外側でないものを描画候補に残す。

05 可視インスタンスを出力バッファ内の DrawType ごとの領域へ詰め直す。

06 インデックス数・インスタンス数・開始インデックスなどの描画引数を記録する。

07 DrawIndexedInstancedIndirect で DrawType 単位に描画する。

08 平行光の視点でも同様にカリングし、影の描画対象を選別する。

技術判断

実装後に検証したところ、対象プロジェクトのオブジェクト数は数千個に満たず、この規模なら CPU 側の視錐台カリングとインスタンシングでも十分で、GPU カリングは必須ではないことが分かりました。

一方で、GPU カリングはすでに問題なく動いており、CPU カリングへ作り直しても得られるものがありません。そのため、現在もこの仕組みをそのまま使っています。新しい技術を入れること自体を目的にせず、規模と要件、作り直すコストまで含めて判断した経験です。

CPU カリングとの性能比較は未計測のため、性能の優劣は記載していません。

エッジシェーダ

2Dアニメ調のビジュアルを目指し、G-buffer に格納した複数の情報から輪郭を検出するエッジシェーダを実装しました。色・法線・オブジェクト識別の3種類のエッジを、それぞれ強度を調整したうえで統合しています。

HLSL | G-buffer | Sobel Filter (3×3) | Edge Detection | Post Process

課題

3Dモデルをそのまま描画した状態では、目指していた2Dアニメ調の見た目に対して画面が平坦に感じられました。そこで、1種類の情報だけに頼らず、色の変化・形の変化・オブジェクトの境界の3つから輪郭を出す仕組みを用意しました。

3種類のエッジ

Color Edge
周りのピクセルとの色の差からエッジを検出する。

Normal Edge
G-buffer の法線を使い、周りのピクセルとの法線の差から形の変化を輪郭にする。

ID Edge
G-buffer にオブジェクトごとの識別値を入れ、周りとの値の差からエッジを検出する。対象外のオブジェクトは -1。考え方は Color Edge と同じで、比べる情報が識別値に変わる。

検出と合成

どのエッジも 3×3 の Sobel フィルタで検出します。Color・Normal・ID にはそれぞれ強度の倍率を持たせ、輪郭の出方を個別に調整できるようにしました。最終的なエッジの強さは次の順で決めます。

01 Color Edge と Normal Edge の大きいほうを取る。

02 その結果と ID Edge の大きいほうを取る。

03 得られた値を最終的なエッジの強さにする。

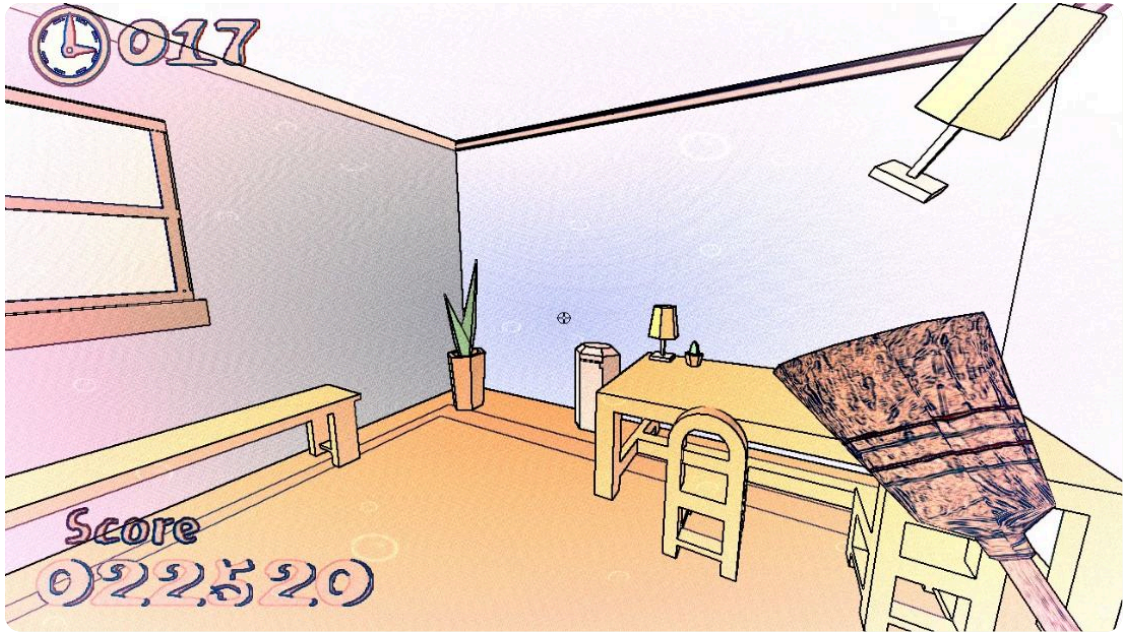
改善の流れ

STEP 1 – **色エッジのみ**
同じ色のオブジェクトが並ぶ場所で、輪郭がのっぺりした。

STEP 2 – 法線エッジを追加
法線の差が大きい場所に線を入れた。不要な線を避けるため、しきい値は大きめに取った。

STEP 3 – IDエッジを追加
ゴミ同士のような同じ種類のオブジェクトの重なりにも線が入り、輪郭が整った。

成果



3種類のエッジを合成した画面。家具どうしの重なりや壁の境目にも輪郭線が入り、平坦だった画面にメリハリが出ています。

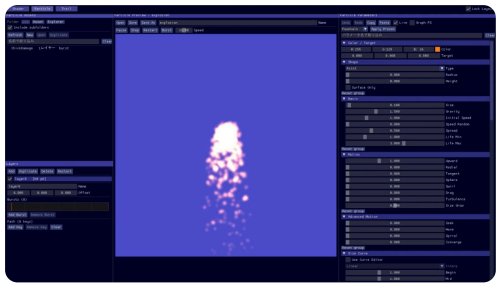
TrashBomb

Effect Editor

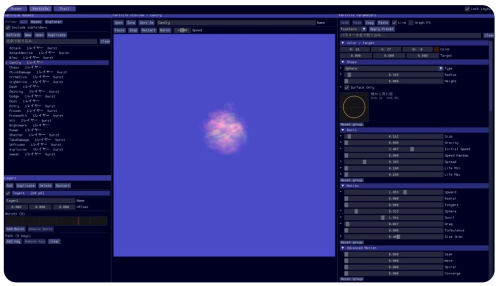
シェーダエディタの機能として、Particle Editor と Trail Editor の2つの独立したエディタを実装しました。パラメータ操作でエフェクトを新しく作れるほか、既存のアセットを読み込んで編集できます。設定は JSON 形式のアセットとして保存・管理します。

C++ JSON Editor Development Data Serialization Particle System Trail System

画面



Particle Editor。左でレイヤーとバーストを管理し、中央でプレビューしながら、右のパラメータで動きを調整します。



BreaveChicken のパーティクルアセット一覧（23種）。必殺技「鳴き声」の CanCry や、凍結の Frozen などをここで作っています。



Trail Editor。斬撃の軌道でプレビューしながら、色のグラデーションや形状、ノイズを調整します（10種）。

Particle Editor

パラメータを調整してパーティクルエフェクトを新しく作れます。既存の Particle アセットを読み込んで編集することもできます。

- アセットはレイヤーを持ち、レイヤーごとにパーティクルの設定を持つ。複数レイヤーを含めたエフェクト全体をプレビューできる。
- バースト設定で、粒子の発生数と起爆タイミングを持つ。起爆タイミングは、ゲームからエフェクトが呼ばれた時点からの経過時間で決まる。
- 画面で調整できる項目：色、発生形状（Sphere・Point など）、サイズ・重力・初速・寿命、動き（上昇・放射・渦など）、サイズとアルファのカーブ、ビルボード、カールノイズ、形状（角の数・丸み・とがり）など。
- Undo / Redo、パラメータのコピー＆ペースト、プリセットの適用に対応。

Trail Editor

Particle Editor とは独立したエディタです。パラメータを調整してトレイルを新しく作れるほか、既存の Trail アセットを読み込んで編集できます。

- プレビューでは、斬撃などの軌道とビルボードの幅モードを選んで確認できる。

- 画面で調整できる項目：寿命・幅・強さ、端のぼかし、UV（引き伸ばし・タイリング・スクロール）、色のグラデーション、芯の太さと明るさ、ノイズ、UV ノイズなど。
- 編集中のトレイルをゲームへ送る操作がある（「ゲームへ送る」「自動送信」）。

アセットの管理と保存

- 保存ボタンか Ctrl ＋ S で、JSON 形式のアセットとして保存する。
- 指定したフォルダからアセットを読み込む。Particle アセットはファイルの中から自動で選び分ける。
- 読み込んだ既存のアセットを、エディタ上でそのまま再編集できる。

データ	内容
名前	Particle アセットの名前
レイヤー	パーティクルを構成するレイヤー（名前・有効状態・オフセット）
バースト設定	粒子数・起爆タイミング
パラメータ	パーティクルの挙動・形状・色・サイズなどの設定
テクスチャ設定	テクスチャ関連の設定

テクスチャを使う設定はありますが、現在のパーティクルは SDF で形を作っているため、通常はテクスチャを使っていません。

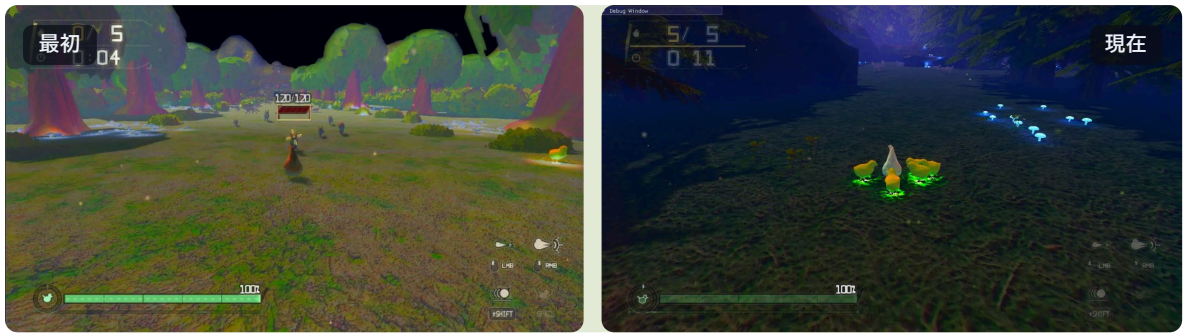
シェーダエディタ　GPUパーティクルシステム

夜の森のルック調整

BreaveChicken のステージを「夜の森の中を進む、神秘的で少し不気味な空間」に近づけるため、明るさ・色調・コントラスト・フォグ・オブジェクト配置・発光表現を6段階で見直しました。1つ直すと別の問題が出るので、画面で確かめながら調整を繰り返しています。

`Color Grading` `Contrast` `Distance Fog` `Emission` `Level Layout`

最初と現在



左がルック調整を始める前、右が現在の画面です。境目をドラッグするか、左右キーで見比べられます。撮影した場面とステージの配置は異なります。

目指した表現

- 夜の森らしい、暗く落ち着いた色調。
- 青・紫を基調とした神秘的な雰囲気。
- 森の奥へ進むほど見え方が変わる奥行き。
- 暗い背景の中でも、プレイヤー・ひよこ・敵を見分けられること。
- キノコなどの発光物が、森の不思議さを出す控えめなアクセントになること。

調整の流れ



初期状態。夜の設定なのに画面全体が明るい。



STEP 1：全体を暗く、水色寄りに。夜らしさは出たが、キャラクターが見えにくい。



STEP 2：コントラストを強めた。見やすいが、キャラクターが背景から浮いている。



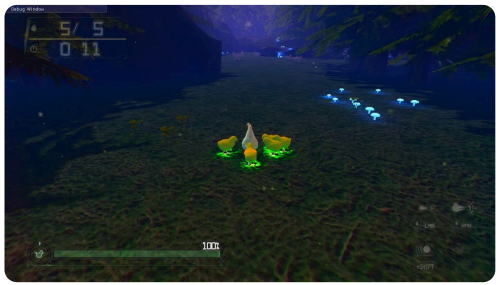
STEP 3：淡い紫でなじませ、木のモデルを変更。画面に統一感が出た。



STEP 4：岩・倒木を置き、距離減衰フォグで奥行きを出した。



STEP 5：森の密度を上げ、道を左右に曲げた。先が見通しにくくなった。



STEP 6（現在）：さらに暗くし、キノコの光を抑えた。

STEP 1 – 全体を暗くする

夜の設定なのに明るかったため、明度と彩度を下げて水色寄りにした。夜らしさは出たが、背景とプレイヤー・敵の区別がつきにくくなった。

STEP 2 – コントラストを強める

背景とキャラクターの明暗差・色の差を強めた。見やすくはなったが、キャラクターが背景から浮き、画面の一体感が弱くなった。

STEP 3 – 色調をなじませ、木を変える

淡い紫を取り入れ、コントラストの差を抑えた。ローポリ調の木も森の雰囲気に合うものに変えた。画面全体に統一感が出た。

STEP 4 – オブジェクトとフォグで奥行きを出す

岩や倒木を置き、遠くほどフォグ色になじむよう距離減衰フォグを調整した。手前と奥を見分けやすくなった。

STEP 5 — **森の密度と道の形を見直す**
木やキノコの配置を増やし、一本道を左右に曲がる形に変えた。青系を強め、森の中を進んでいる感覚が強まった。

STEP 6 — **暗さと発光を再調整する**
全体をさらに暗くし、キノコの発光を抑えた。光が画面を支配せず、暗闇に点在する控えめな光になった。

色の役割

色・要素	画面での役割
青・紫	夜の森、遠景、空間全体の雰囲気
緑	地面や植物、キャラクターの足元の光
オレンジ	ひよこのシルエットを背景から見分けるための色
白	プレイヤーのニワトリを見分けるための色
キノコの光	暗い森に点在する、神秘的なアクセント

フォグと発光

距離減衰フォグは、遠くのオブジェクトほどフォグ色に近づくように調整しました。距離の減衰が足りなかったため、ワールド座標を基準に距離を扱う方式に見直しています。

キノコの発光は、明るければよいわけではありません。強すぎると暗い森の雰囲気を壊し、光源ばかりが目立つため、強さを抑えて環境のアクセントとして扱いました。ニワトリとひよこの足元の緑の光は、暗い画面でキャラクターの位置をつかむ手がかりにもなっています。

わかったこと

1つの問題を直すと、別の問題が出ることを何度も確かめました。

- 暗くすると夜らしさは増すが、キャラクターが見えにくくなる。
- コントラストを強めると見やすくなるが、背景から浮いて見える。
- 色調をそろえると統一感は増すが、キャラクターとの区別を保つ必要がある。
- フォグや配置を変えると、画面の印象だけでなく、奥行きや進む方向の感じ方も変わる。

- 発光を強めると目立つが、強すぎると環境全体の雰囲気を損なう。

> 効果を1つずつ強くするのではなく、画面全体の色調・視認性・空間表現のバランスを見ながら調整することが大切だと学びました。

今後の改善

- 地面の視認性：全体を明るくせず、地面のライティングや局所的な明暗を見直して、テクスチャや凹凸の情報を残す。
- キャラクターの輪郭：背景から不自然に浮かない範囲で、輪郭や明暗差を調整する。
- 遠景の情報量：遠景を黒く潰さず、フォグの奥行きを保ちながら森の存在を感じられるようにする。

BreaveChicken 描画パイプライン設計

© 2026 ニック

最終更新日 2026-11-06